TAPLEND LOAN PREDICTION

Problem statement, code walkthrough, results and limitations

Project aim	Use past loan application information and loan outcomes to estimate whether a loan is likely to be good or bad.
Inputs reviewed	Taplend_loan_prediction notebook, application data, loan performance data, and assignment brief.
What this report covers	The business problem, data preparation, exploratory analysis, model training, measured performance, and practical caveats.

Prepared from the supplied notebook and spreadsheets. Results below reflect the outputs recorded in the notebook; they are not a guarantee of future lending performance.

1. Problem statement

The assignment provides two datasets covering loans issued over a six-month period. One contains application-level information about customers and the other records whether each loan was classified as **Good** or **Bad**. The task is to use the historical examples to identify which future applicants may be suitable for a loan, while explaining the limitations of the data and analysis.

In practical terms, this is a **binary classification** problem. The model learns patterns from previously labelled loans and estimates the likelihood that a new application belongs to the Good-loan class. That estimate can support a review process, but it should not be treated as an automatic or final lending decision.

Main questions the notebook tries to answer

- Can the application and credit-score fields help distinguish good loans from bad loans?
- How do outcomes vary across categories such as payment frequency, residence history and bank-account history?
- Which classification approach performs best on held-out data?
- What are the model's weaknesses, and what additional checks would be needed before real-world use?

2. Data used

The supplied spreadsheets contain the following structures:

File	Rows × columns	Fields used
Application data	647 × 31	Customer identifiers, requested/approved amounts, dates, income, rent, repayment details, account/residence information, and raw credit scores.
Loan performance	1,285 × 2	Loan identifier (idLoan) and outcome label (flgGood).

The notebook derives a customer_id from the text before the hyphen in idLoan, converts it to lowercase, and joins the two tables on customer_id. After this merge, the notebook reports 650 rows and 34 columns. It creates the target variable is_good, where Good is encoded as 1 and other labels as 0.

Important data check: the merged row count (650) is larger than the application table's 647 rows. This may be caused by multiple matching loan records or the join key, and should be investigated before trusting the model. The notebook reports duplicate idLoan count, but that alone does not prove that customer-level matching is correct.

3. What happens in the code

The notebook is organised as a standard data-science workflow. Each stage prepares the data for the next one.

Step 1 — Import libraries and load files

Pandas and NumPy handle tables and numerical values. Matplotlib and Seaborn produce charts. Scikit-learn supplies data splitting, preprocessing pipelines, classification models and evaluation metrics. Joblib is used to save the final fitted pipeline. The two Excel files are read into separate DataFrames, and their dimensions and sample rows are inspected.

Step 2 — Join the labels to applications

The code extracts a customer identifier from idLoan and performs an inner join with the application table. An inner join keeps only records that match on both sides. The target is created using (flgGood == 'Good').astype(int), so Good becomes 1 and Bad becomes 0. The notebook's merged target distribution is about 45.7% Good and 54.3% Bad.

Step 3 — Check missing values and summaries

The notebook counts null values, prints descriptive statistics, and checks repeated loan IDs. These checks help reveal missing fields, unusual ranges and potential duplication. Missing values are then filled in selected columns: for example, missing payment_amount_approved values use payment_amount, while missing how_use_money and bank_account_duration values are labelled Unknown.

Step 4 — Create extra features

Age is calculated from application_when minus birth_date in years. Three ratios are added: loan_to_income (approved amount divided by monthly income), payment_to_income (payment amount divided by monthly income), and rent_to_income (monthly rent divided by monthly income). Zero income is replaced with a missing value before the divisions to avoid dividing by zero. The code also fills missing other_phone_type values with None.

Step 5 — Explore patterns (EDA)

- Plot counts of Good and Bad loans.
- Calculate ROC-AUC for columns whose names start with raw_ to compare the credit-score fields individually.
- Compare Good-loan rates across categories such as residence, direct deposit, ACH payments, payment frequency and account duration.
- Compare numerical features by outcome using medians and boxplots; group income, age and raw_l2c_score into quartiles.
- Check correlations between numerical features and the target, and chart monthly Good-loan rates.

This section is descriptive: a relationship in a chart does not, by itself, show that a feature causes a loan outcome.

4. Model training and evaluation

The notebook first prepares a modelling table by excluding identifiers, the target label and some fields such as birth_date, application_when, email, bank_routing_number, address_zip, status and month. It then builds a preprocessing-and-model pipeline so that missing values and categorical fields can be handled consistently.

Preprocessing

- Numeric columns: missing values are replaced with the median. Logistic Regression additionally scales numeric fields using StandardScaler.
- Categorical columns: missing values are replaced with the most frequent category, then OneHotEncoder converts categories into model-readable columns. Unseen categories are ignored.

Three models are compared

Model	ROC-AUC	Avg. precision	Accuracy	Precision	Recall	F1
Logistic Regression	0.6833	0.6892	0.6154	0.5843	0.8000	0.6753
Random Forest	0.6690	0.6317	0.6231	0.6333	0.5846	0.6080
Extra Trees	0.6250	0.6146	0.5615	0.5667	0.5231	0.5440

In this first comparison, Logistic Regression has the highest ROC-AUC (0.6833) and recall (0.8000), while Random Forest has the highest accuracy (0.6231) and precision (0.6333). There is no single metric that captures every business cost, so model selection should depend on the consequences of incorrectly approving a risky loan versus incorrectly rejecting a good applicant.

Tuning the Extra Trees model

The code then runs RandomizedSearchCV for an Extra Trees pipeline, testing 20 parameter combinations with five-fold stratified cross-validation. It selects settings using ROC-AUC. The recorded best cross-validation ROC-AUC is 0.6318. On the held-out test set, the tuned model reaches ROC-AUC 0.6563 and average precision 0.6382. At a 0.5 probability threshold, accuracy is about 0.58.

Test-set classification details

Class	Precision	Recall	F1-score	Support
Bad Loan	0.60	0.49	0.54	65
Good Loan	0.57	0.68	0.62	65

The final model's Bad-loan recall of 0.49 means it identifies roughly half of the bad loans in this test set at the chosen threshold. That is a material limitation for a lending-risk use case.

5. Final outputs and interpretation

After tuning, the notebook displays a classification report and confusion matrix, ranks the 20 most important transformed features, and saves the fitted pipeline as taplend_loan_pipeline.joblib. The top recorded importance is payment_frequency_Bi-weekly, followed by payment_frequency_Semi-monthly and residence_duration_7-12 months. Other high-ranked fields include raw_FICO_retail and amount_requested.

Feature importance indicates which variables the fitted tree model used most in its splits. It does not establish that these variables cause repayment outcomes, nor does it prove that they are fair, stable or appropriate decision criteria.

The final cell predicts labels and probabilities for a separate table called X_new. That variable is not created in the supplied notebook cells, so this last step will only run if X_new has already been prepared in the notebook session with the same input columns expected by the fitted pipeline.

6. Limitations and issues to address

- **Small dataset:** only 650 merged rows are reported. A small sample can produce unstable estimates and may not represent future applicants.
- **Join validation:** the loan table has 1,285 rows and the application table 647, while the merge returns 650 rows. Confirm whether the extracted customer_id uniquely and correctly maps each loan to its application.
- **Conflicting modelling preparations:** an earlier cell creates X using get_dummies and median filling, but the later model comparison uses a separate X_model pipeline. The latter is the route used for the reported model results.
- **Time-based split:** the code sorts applications by date and uses the earliest 80% for training and latest 20% for testing, if dates are available. This is more realistic than a random split for future prediction, but only six months of history may be too short to test changes over time.
- **Limited predictive strength:** tuned test ROC-AUC of 0.6563 suggests only moderate separation between the classes. Results are not strong enough to justify unreviewed automatic lending decisions.
- **Threshold and costs:** 0.5 is used as the decision threshold without an explicit analysis of the financial costs of false approvals and false rejections.
- **Potential leakage or unsuitable fields:** verify that every feature is genuinely known at application time and is not recorded after approval or outcome. Review sensitive proxies, fairness, privacy and applicable lending rules.
- **No external validation:** there is no independent later-period or out-of-sample validation beyond the notebook's test split, and no monitoring plan is shown.
- **Data-quality handling:** median imputation and category filling are practical starting points, but missingness patterns and outliers need investigation. The earlier exploratory feature construction also divides by zero-income-as-missing, which can produce missing ratios.

7. Recommendations

- Audit the customer/loan join, duplicate records, label definitions and date coverage before retraining.
- Confirm that all predictors are available at the moment a new application is assessed; remove post-outcome or post-decision information.
- Evaluate business-relevant thresholds using expected loss, bad-loan capture and approval rate, not accuracy alone.
- Validate on a later time period and report confidence intervals or repeated-split variability.
- Compare the tuned model against a simple baseline and the Logistic Regression result; do not assume the tuned Extra Trees model is best just because it was tuned.
- Review fairness, explainability, governance and human oversight before any production use.

Conclusion

The notebook demonstrates a complete first-pass classification workflow: it combines application and outcome data, creates derived features, explores patterns, compares three models, tunes an Extra Trees classifier and saves the resulting pipeline. The recorded results show some predictive signal, but performance is moderate and the dataset is small. The most important next step is to verify the data join and validate the model more thoroughly before using its predictions to guide real lending decisions.

Source note: Figures and model metrics in this report are taken from the supplied notebook outputs. Where the notebook does not provide evidence (for example, a fairness assessment or a defined cost threshold), the report identifies this as a limitation rather than assuming a result.